

Qt frontend: RAM consumption

Track N Race · Qt 6.11.1, Direct3D 11 · measured 8 October 2026

Summary

- **Live, 15.5 minutes at 1x with every page visited: 383 MB private at the end.** It starts at 187 MB, is about 256 MB once the race is running and 276 MB at minute 5, before the page tour. Page switches add short spikes of 50–100 MB (peak 454 MB).
- **A whole race live, on Overview with Current Lap: 444 MB at the end, peak 463 MB** (section 5).
- **End of the race, All Laps on every page: 570 MB after the tour, peak 712 MB.** Before the changes made this week the same run ended at 789 MB with a 1,131 MB peak. Electron on the same run: 643–808 MB, peak 1,115 MB.
- **The telemetry itself is the small part.** At minute 15.5 the session data is 11.8 MB and the chart points 8.0 MB. At the end of the race (playback) they are 71 MB and 24 MB, and every sample is kept.
- **Most of the memory is fixed cost.** Graphics-driver memory outside the heaps is 150–190 MB. Qt, the widgets and caches take 100–250 MB of heap. The engine (recording writer and Strategy) takes about 70 MB when live.
- **Direct3D 11 is now the Windows default.** It idles about 155 MB lower than Vulkan on this machine.

Windows 11, RTX 4050 laptop GPU, 1920 × 1080 at 125 %. Data: the 92-minute São Paulo race, as the UDP capture (`udp_capture.bin`, replayed with `play_udp.py`) or as its recording (`udp_capture_sao_paulo_v6_2026-10-07.tnrd`). **Private memory** is memory only this program uses; it is the number to compare. The **working set** is what currently sits in physical RAM, and Windows trims it freely.

1. Live session

The UDP capture replayed at 1x into the app. The app stays on Overview for 5 minutes, then visits every page for 40 s and goes back to Overview. Chart window: Current Lap. One sample a second from the app's own memory log (`ram_usage.log`, run 13).

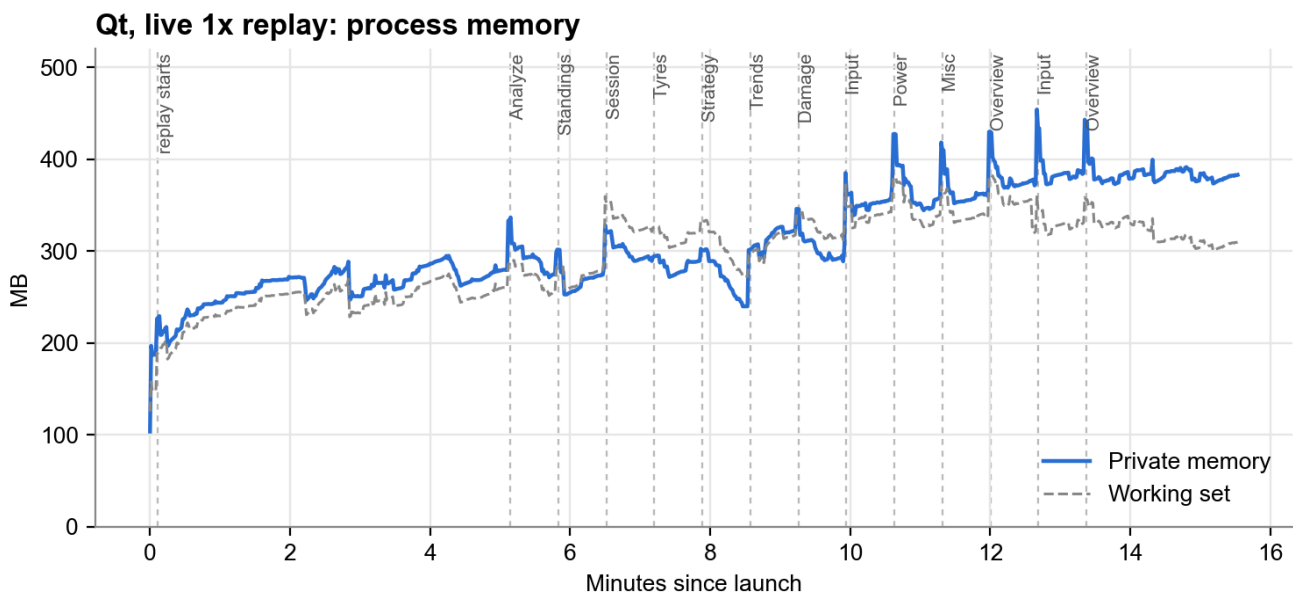


Figure 1. Process memory through the live run. Each dashed line is a page switch.

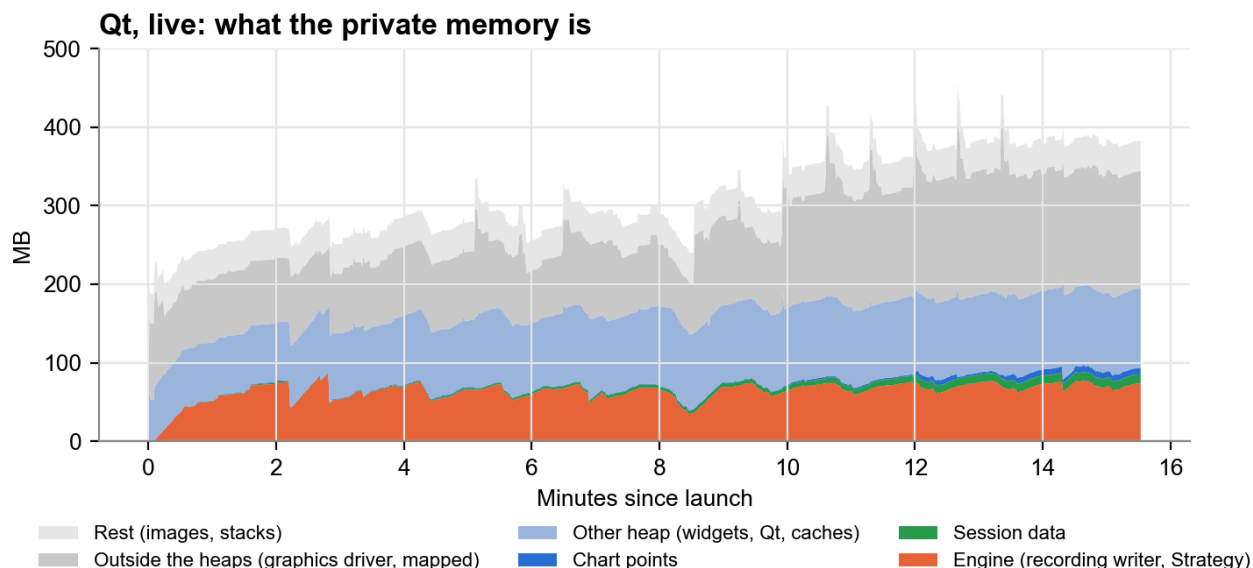


Figure 2. The same private memory split by owner. The layers add up to the total. Engine, session data and chart points come from the app's own counters; the rest is from Windows' heap and memory-region queries.

At	Private	Engine	Session data	Chart points	Other heap	Outside heaps	Rest
Start, no data	187	0	0	0	53	97	38
Minute 1 (race running)	244	51	0.7	0.4	73	81	38
Minute 5 (before the tour)	279	66	1.9	1.3	84	88	38
Minute 10 (mid-tour)	362	64	6.2	1.5	98	154	39
Minute 15.5 (end)	383	73	11.8	8.0	102	149	39

All figures in MB. The session data grows by about 0.8 MB a minute (11 laps at the end of the run). Chart points are only held for the page on screen. Everything else is close to flat once each page has been opened once.

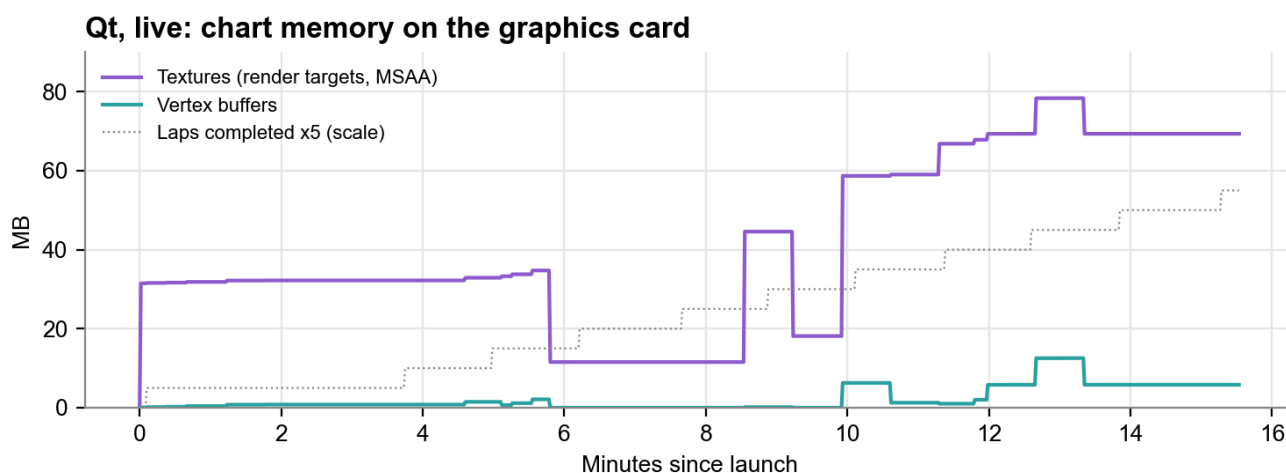


Figure 3. Chart memory on the graphics card, which is not part of the private figure. It follows the page on screen: textures are the charts' render targets with 4x MSAA, and vertex buffers hold the visible charts' points.

2. Graphics backend

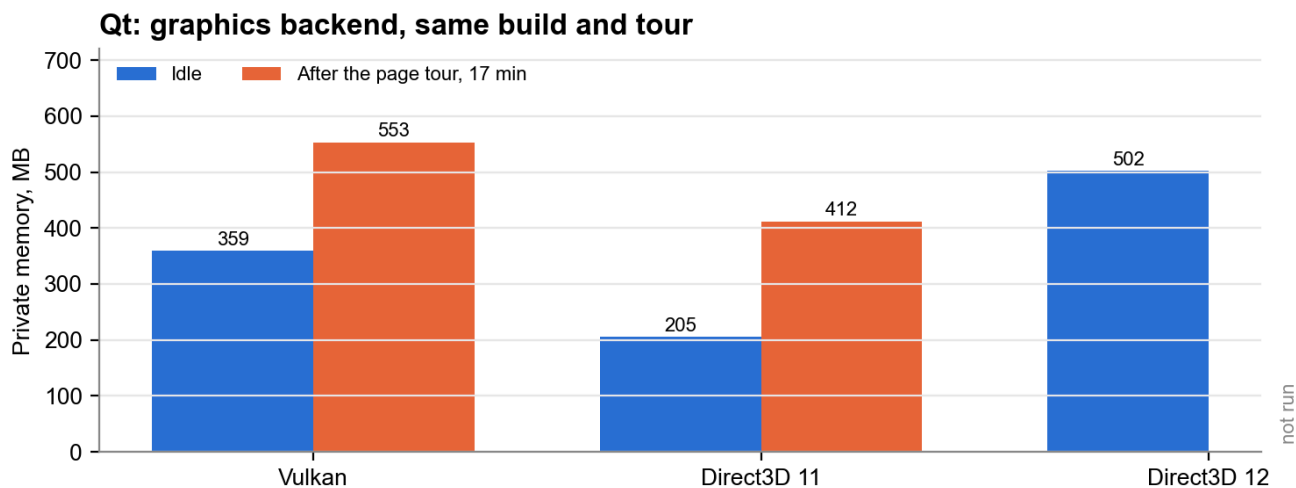


Figure 4. Same build and page tour on each backend (runs 3 and 4). Direct3D 12 was only measured idle.

Vulkan also allocated 130–190 MB outside the heaps on some page switches and returned only part of it. Direct3D 11 doesn't, so it is now the default on Windows; Vulkan and Direct3D 12 stay selectable in Settings.

3. End of the race, All Laps

The recording, seeked to minute 88 and played at 1x with All Laps on every page, visiting Input, Power, Misc, Tyres, Trends, Analyze and Overview for 41 s each. This is the heaviest case: every chart holds the whole race. Playback stands in for a full live race here (same charts, same sample counts). Its session data and engine differ from live.

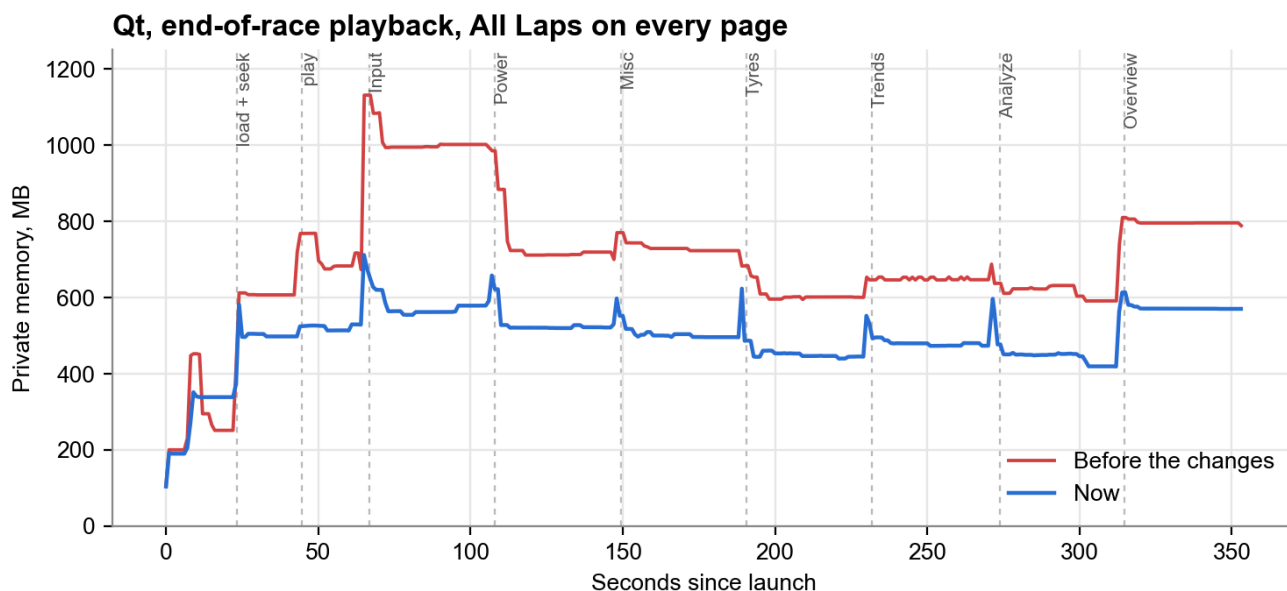


Figure 5. The same tour before and after this week's changes.

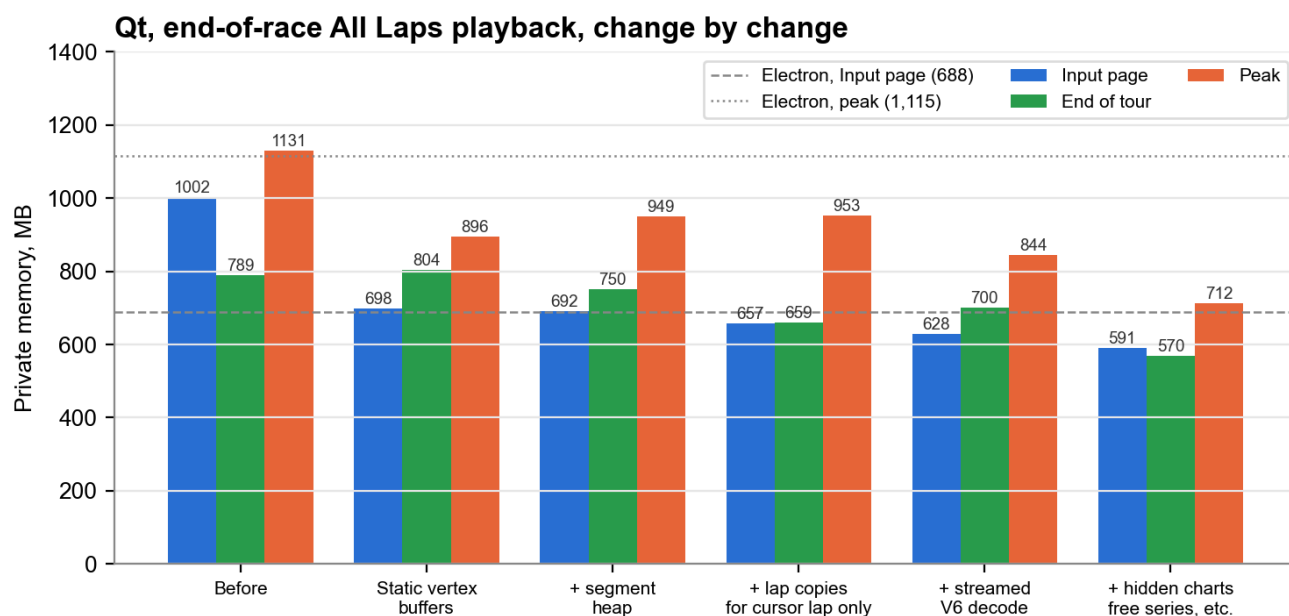


Figure 6. Each change's effect on the same run. Run-to-run variation is about ± 40 MB, so steps smaller than that are not significant on their own.

Change	What it does
Static vertex buffers	On Direct3D 11, Qt kept a CPU copy of every dynamic vertex buffer, and the driver kept more.
Segment heap	Windows' newer allocator, chosen in <code>app.manifest</code> . Lower settled memory; higher transient peaks.
Lap copies for the cursor's lap only	Seek and window history was stored twice, in the session buffers and in every lap block.
Streamed V6 decode	History fields stay packed and are read in order, instead of two temporary tables of doubles.
Hidden charts free their series, shared x columns, slimmer tyre samples	A chart on a hidden page holds no points; series with the same keys share one key column; tyre samples dropped 16 bytes of unread wear.

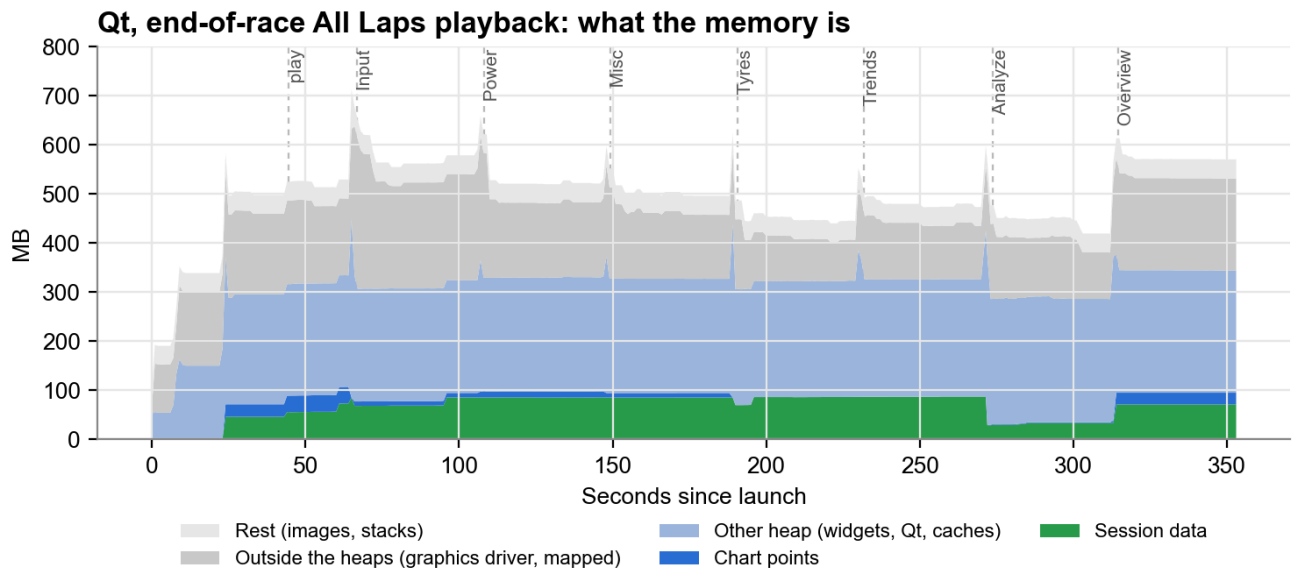


Figure 7. What the memory is during the end-of-race tour. In playback the engine reports no figures, so its memory sits in “Other heap”.

At the end of the tour	MB
Private memory	570
Session data (every sample of the race)	70.7
Chart points (Overview)	24.0
Other heap: Qt, widgets, the recording reader's chunk cache (up to 64 MB), recording load (~86 MB)	249
Outside the heaps: graphics driver, mapped memory	187
Rest: program images, thread stacks	39
On the graphics card (not in private): textures / vertex buffers	103 / 46

4. All Laps against Current Lap

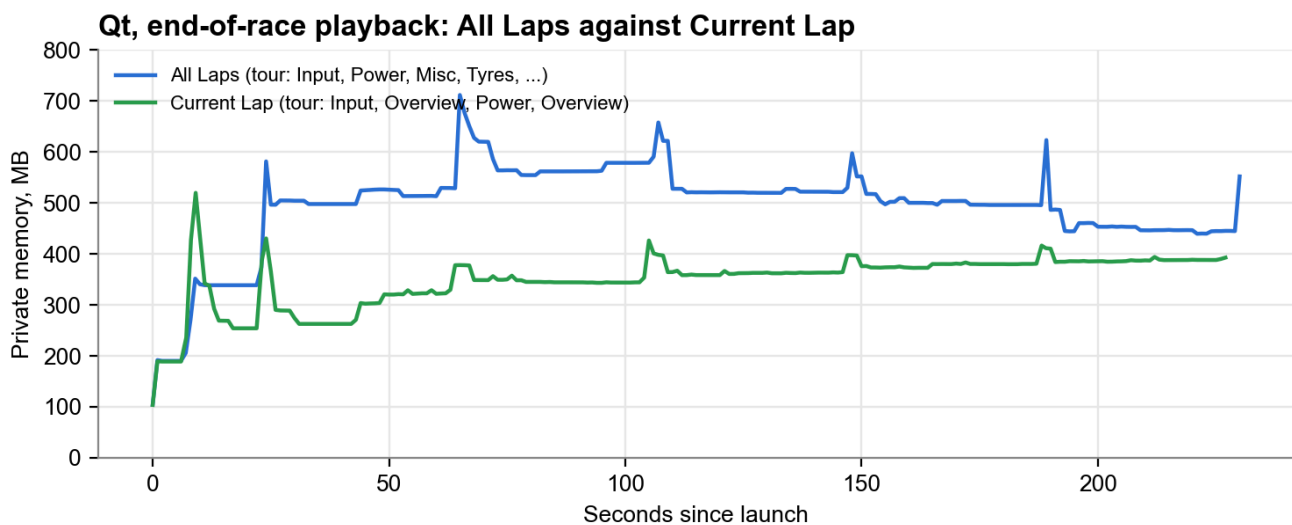


Figure 8. The same recording and seek point with each chart window. The two runs visit different pages after Input, so compare the levels, not each step.

Current Lap holds about 390 MB at the end of its tour, against 450–570 MB for All Laps. The difference is the whole-race history the All Laps charts hold, plus their bigger point sets.

5. A whole race, live

On 9 October Qt ran the whole capture live at 1x, fresh from launch: 99 minutes, the formation lap and 71 laps, on Overview with Current Lap, recording the race to a file as it does by default. Nothing else was touched.

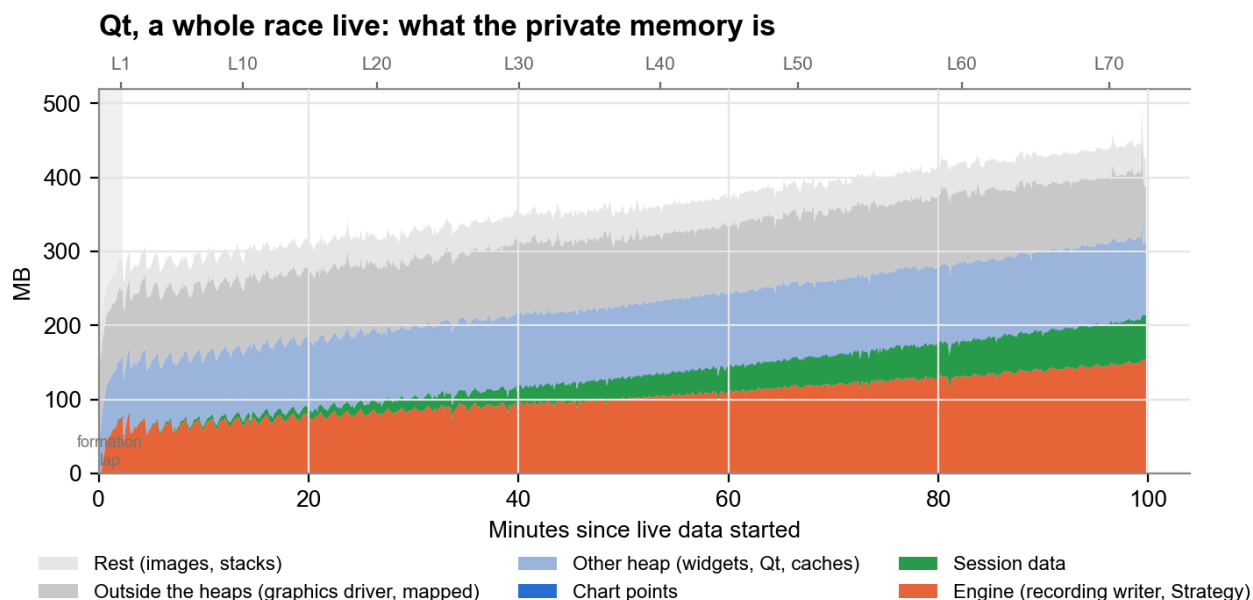


Figure 9. Qt's private memory over the whole race, split by owner, from its own memory log (one sample a second). Lap numbers run along the top. The layers add up to the total.

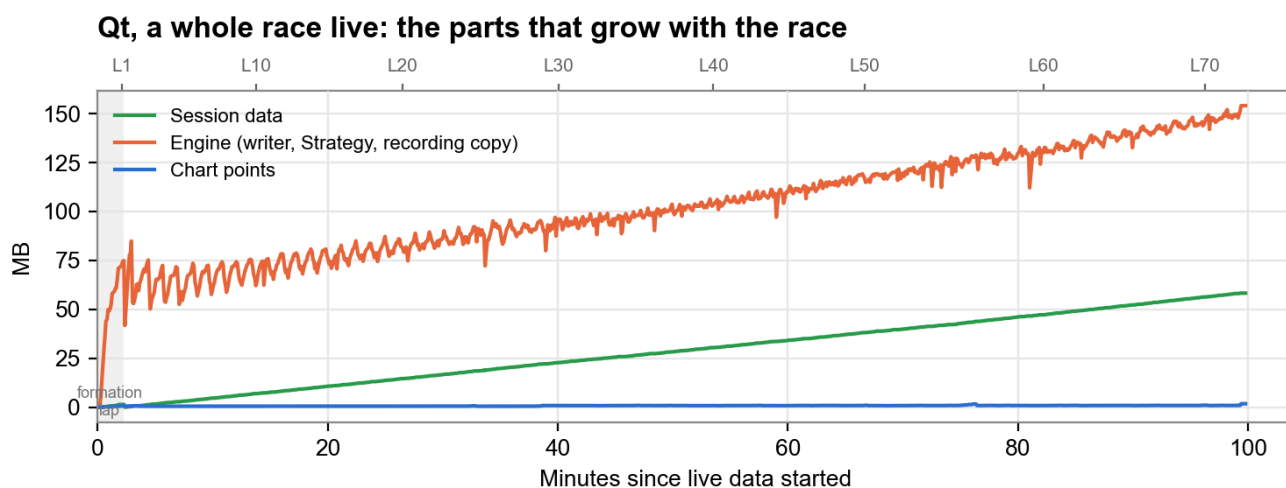


Figure 10. The parts that grow with the race. The engine's growth is mostly its in-memory copy of the recording; the session data is every sample of the race; chart points are only the page on screen.

	Private	Engine	Session data	Chart points	Other heap	Outside heaps
Minute 2	282	71	1	0.6	78	94
Minute 30	328	86	17	0.5	95	92
Minute 60	372	110	34	0.8	99	90
End of the race	445	150	58	1.0	109	88

Qt ended the race at 444 MB, with a peak of 463 MB, and grew about 103 MB an hour over the last hour. That growth is the engine (71 → 150 MB, mostly the in-memory copy of the recording, next step 1) and the session data (0 → 58 MB, every sample of the race). Everything else (Qt, the widgets, the graphics driver) stays at 210–240 MB. With All Laps the charts would hold the whole race too; the end-of-race All Laps playback in section 3 reached 570 MB.

6. What's proven and what isn't

Claim	Status
Live: 383 MB private after 15.5 minutes with every page visited	Measured (run 13)
End of the race, All Laps: 570 MB at the end of the tour, peak 712 MB	Measured (final playback run)
Electron: 643–808 MB on the same playback run, peak 1,115 MB	Measured (same tour)
Session data grows about 0.8 MB a minute live	Measured: 58 MB at the end of a whole race (section 5)
Each change's saving in Figure 6	Measured, one run each; ±40 MB run-to-run
A whole race live (Overview, Current Lap): 444 MB at the end, peak 463 MB	Measured (9 October, one run)
What the ~100–250 MB of “Other heap” is, item by item	Not attributed beyond the reader cache and load figures
Behaviour on Vulkan or on other GPUs	Only this laptop; Vulkan figures are from before the changes

7. Next steps for optimization

Ordered by expected saving for the effort. None of them drops or thins out data. **Saving** says whether the figure is measured, computed or an estimate.

#	Step	Where	Saving	Effort and risk
1	Keep the recording's chunks on disk while a file holds them	Engine: TNRD_V6.cpp, LiveV6Store.cpp	11 MB at 15 min, 81 MB at the end of a race (measured)	Medium. Reads come from a file still being written
2	Shrink Strategy's rollback journal	Engine: StrategyRollback.h	Up to ~25 MB of the 31 MB (measured size)	Low. Deep flashbacks rebuild from the store instead
3	Squeeze each lap's vectors when the lap closes	SessionModel.cpp, finalizeCurrentLap	Roughly 10–20 MB by race end (estimate)	Low
4	Size the writer's lap builders to what they hold	Engine: V6 writer builders	Up to ~11 MB (20.3 MB capacity for 8.9 MB pending, measured)	Low
5	Antialias lines in the shader instead of 4x MSAA	ChartView.cpp, render target and line shaders	~26–34 MB per chart page at 1080p (computed)	Medium. Small visual change on fills and discs
6	Upload chart series in pieces on page switches	ChartView.cpp, series upload	Lower page-switch peaks (part of the 50–140 MB spikes)	Low–medium
7	Playback: smaller reader cache; account for the ~86 MB taken at load	Engine: TNRD_V6.cpp (cacheLimit, 64 MB)	Up to ~32–48 MB in playback (estimate)	Low. Seeks may decode more often; measure
8	Attribute the 100–250 MB of “Other heap” before acting on it	Heap profiling (ETW heap trace or UMDH)	None by itself; finds the next target	Low
9	One render surface per page instead of one per chart	ChartView, page layouts	Shared render target and driver state (not estimated)	High. Large refactor

Notes

- **Recording chunks on disk (1).** Qt runs the engine with `binaryPlayback` on (`MainWindow.cpp`), so the live V6 store keeps every committed chunk in memory (`memoryPayloads` in `TNRD_V6.cpp`, `appendEncoded`) and also appends it to the recording file. At minute 15.5 that was 11.1 MB in 5,290 chunks. At the end of the whole race it was 70 MB of chunks plus 11.5 MB of shared records (measured, 9 October). While a file is attached, the store could keep only each chunk's file offset and read it back from the file when a page needs history. Without a recording file it has to stay in memory. This change is shared with Electron.
- **Strategy's journal (2).** The rollback journal sat at its 32 MB cap (`StrategyRollback.h`, `MAX_BYTES`) with nothing subscribed to Strategy. A flashback deeper than the journal already rebuilds Strategy from the live store (`Engine.cpp`, around line 640), so a smaller cap trades memory for slower deep flashbacks. That CPU cost is not measured.
- **Lap vectors (3).** `SessionData::finalizeCurrentLap` pushes the finished lap with whatever capacity its vectors grew to. Qt grows them geometrically, so a finished lap keeps unused room, on average about a quarter (estimate). One `squeeze()` per family when the lap closes returns it.
- **Page-switch spikes (6).** Showing a chart page rebuilds its series from the session data and uploads each one in a single staging array, which the driver copies again. Uploading in fixed-size pieces, or writing vertices straight into the buffer, would cap that transient. The 50–140 MB spike size is measured; how much of it this accounts for is not.

- **One surface per page (9).** Every chart is its own `QRhiWidget` with its own colour texture and MSAA buffer; 8 canvases existed at the end of the live run, 2 of them on screen. One canvas per page would share one target, but it is a large change to `ChartView`.

8. Observations

Overall: good. Qt now uses less memory than Electron in every case measured on both (idle and the same All Laps tour), keeps every sample, and its own telemetry is a small, predictable share of the total.

- **The telemetry costs what it should.** A whole race of session data is about 71 MB, and chart points 24 MB for the page on screen. Both grow linearly with race length, and the duplicate copies are gone. Apart from the growth slack finished laps keep (step 3), there isn't much more to win here without dropping data, which is ruled out.
- **The fixed cost is now the bigger target.** Driver memory outside the heaps (150–190 MB) and unattributed heap (100–250 MB) are over half of the total in every run. They are set by Qt, the driver and the widgets, not by the race, so they don't grow with it.
- **The page-switch spikes are worth watching.** Opening a chart page briefly adds 50–100 MB live and up to 140 MB in All Laps playback, while it builds render targets and uploads points. A machine short on RAM sees the peak, not the average.
- **MSAA is the largest single chart cost left.** At 4x it is 16 bytes a pixel per visible chart, about 26–34 MB per chart page at 1080p. Antialiasing the lines in the shader would remove most of it, with a small visual change.
- **The engine's ~70 MB is the same code in all three apps.** About 31 MB is Strategy's rollback journal at its cap. The rest is the recording writer: per-lap buffers for 22 cars, needed until each lap is written, and an in-memory copy of everything already written, which grows with the race (steps 1, 2 and 4).
- **A whole race live is now measured.** Over 99 minutes Qt grew about 103 MB an hour, all of it engine and session data, and ended at 444 MB. The in-memory copy of the recording (step 1) is the largest single item that grows with the race.